



TAG'D System Design Document

04.30.2026

Team Beta

Wesley Icer

Isabel Acosta

Jonathan Elia

Caleb Abraham

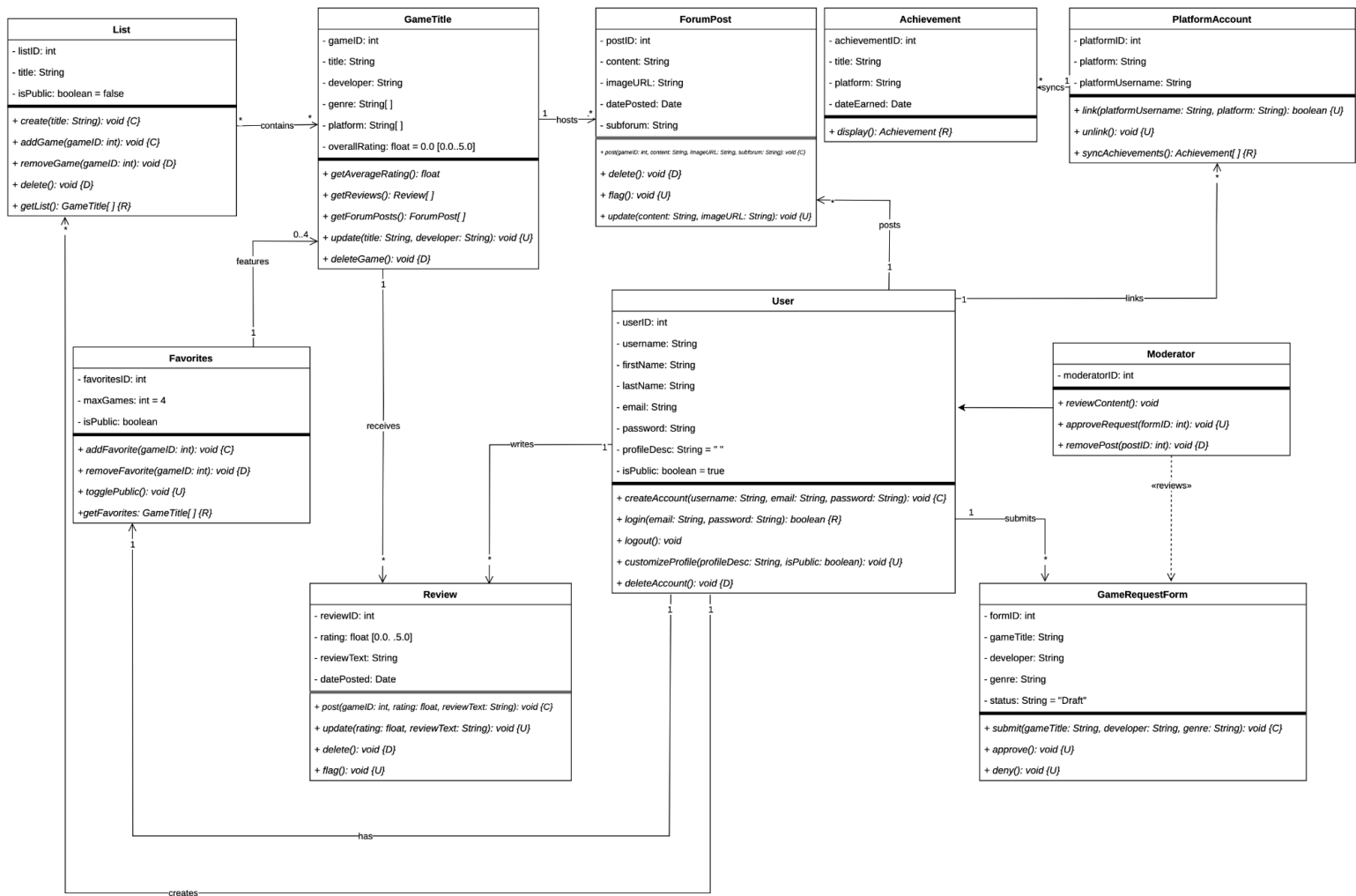
Table of Contents

Table of Contents	1
Overview	2
Design-Level Class Diagram	3
Pseudocode (.txt file is in GitHub)	4
Statechart Diagrams	9
List	9
Favorites	9
Review	10
User	10
GameTitle	10
ForumPost	11
PlatformAccount	12
Achievement	12
Moderator	12
GameRequestForm	13
First Cut Domain Sequence Diagrams	14
Rating/Reviewing	14
Game Forum Post	15
Favorites	16
Game Request Form	17
Account CreationList creation	18
Achievement Sync	19
Account Customization	20
Moderator	21

Overview

The purpose of this System Design Document is to translate the project requirements into concrete technical designs, serving as a bridge between analysis and implementation. TAG'D is a social media and cataloging platform for video games where users can create accounts, search for games, rate and review titles, build personalized lists, favorite games, and interact through forum posts. This document includes design-level class diagrams, statechart diagrams, and first-cut domain sequence diagrams that define the system's structure, component relationships, and key user interactions. Together, these artifacts provide the technical foundation for how TAG'D will be organized and developed.

Design-Level Class Diagram



Pseudocode (.txt file is in GitHub)

TAG'D DESIGN LEVEL CLASS DIAG. - METHOD PSEUDOCODE

Classes: User, Moderator, GameTitle, Achievement, PlatformAccount, List, Favorites, Review, ForumPost, GameRequestForm

CLASS User

```

createAccount(username, email, password)
    VALIDATE username is not empty
    VALIDATE email is valid format
    VALIDATE password length >= 8
    userID      = new auto-generated int
    this.username = username
    this.email   = email
    this.password = hashPassword(password)
    profileDesc = ""
    isPublic    = true
login(email, password)
    IF email is empty OR password is empty -> throw error
    user = look up user by email
    IF user not found -> return false
    IF hashPassword(password) does not match stored hash -> return false
    create session token for user
    RETURN true
logout()
    IF no active session -> return
    invalidate current session token
    clear session data
    RETURN
customizeProfile(profileDesc, isPublic)
    IF profileDesc is null -> throw error
    IF isPublic is null -> throw error
    this.profileDesc = profileDesc
    this.isPublic    = isPublic
    save changes to database
    RETURN
deleteAccount()
    delete all Reviews authored by this user
    delete all ForumPosts authored by this user
    delete all Lists owned by this user
    delete associated Favorites record
    delete all linked PlatformAccounts
    delete this User record from database
    RETURN
  
```

CLASS Moderator

```

reviewContent()
    fetch all flagged Reviews and ForumPosts from database
    IF queue is empty -> return
  
```

```

    display each flagged item for moderator review
    RETURN
approveRequest(formID)
    IF formID is null -> throw error
    form = look up GameRequestForm by formID
    IF form not found -> throw error
    IF form.status is not "Submitted" -> throw error
    form.status = "Approved"
    create new GameTitle using form.gameTitle, form.developer, form.genre
    save new GameTitle to database
    save form changes to database
    RETURN
removePost(postID)
    IF postID is null -> throw error
    post = look up ForumPost or Review by postID
    IF post not found -> throw error
    delete post from database
    notify post author of removal
    RETURN

```

CLASS GameTitle

```

getAverageRating()
    reviews = fetch all Reviews where gameId = this.gameID
    IF reviews is empty -> RETURN 0.0
    sum = 0
    FOR each review in reviews
        sum = sum + review.rating
    RETURN sum / count(reviews)
getReviews()
    reviews = query database for Reviews where gameId = this.gameID
    RETURN reviews
getForumPosts()
    posts = query database for ForumPosts where gameId = this.gameID
    RETURN posts
update(title, developer)
    IF title is empty -> throw error
    IF developer is empty -> throw error
    this.title = title
    this.developer = developer
    save changes to database
    RETURN
deleteGame()
    delete all Reviews for this gameId
    delete all ForumPosts for this gameId
    remove this game from all Lists and Favorites
    delete this GameTitle record from database
    RETURN

```

CLASS Achievement

```

display()
    load achievement details from database using achievementID
    IF achievement not found -> throw error

```

RETURN this Achievement

CLASS PlatformAccount

link(platformUsername, platform)

IF platformUsername is empty -> throw error

IF platform is empty -> throw error

send authentication request to platform API

IF authentication fails -> RETURN false

this.platformUsername = platformUsername

this.platform = platform

save new PlatformAccount record to database

RETURN true

unlink()

IF no linked account exists -> return

revoke platform API access token

delete this PlatformAccount record from database

RETURN

syncAchievements()

IF this.platformUsername is empty -> throw error

rawAchievements = call platform API to fetch achievements for platformUsername

IF API call fails -> throw error

FOR each achievement in rawAchievements

IF achievement not already in local database

create new Achievement record

save to database

RETURN Achievement[]

CLASS List

create(title)

IF title is empty -> throw error

listID = new auto-generated int

this.title = title

this.isPublic = false

save new List record to database

RETURN

addGame(gameID)

IF gameID is null -> throw error

game = look up GameTitle by gameID

IF game not found -> throw error

IF gameID already in this list -> return (no-op)

add gameID to this list

save changes to database

RETURN

removeGame(gameID)

IF gameID is null -> throw error

IF gameID not in this list -> return (no-op)

remove gameID from this list

save changes to database

RETURN

delete()

remove all game references from this list

delete this List record from database

```

    RETURN
getList()
    games = query database for all GameTitles referenced in this list
    RETURN games

```

CLASS Favorites

```

addFavorite(gameID)
    IF gameID is null -> throw error
    IF count(favorites) >= maxGames -> throw error
    game = look up GameTitle by gameID
    IF game not found -> throw error
    IF gameID already in favorites -> return (no-op)
    add gameID to favorites
    save changes to database
    RETURN
removeFavorite(gameID)
    IF gameID is null -> throw error
    IF gameID not in favorites -> return (no-op)
    remove gameID from favorites
    save changes to database
    RETURN
togglePublic()
    this.isPublic = NOT this.isPublic
    save changes to database
    RETURN
getFavorites()
    games = query database for all GameTitles referenced in favorites
    RETURN games

```

CLASS Review

```

post(gameID, rating, reviewText)
    IF gameID is null -> throw error
    IF rating < 0.0 OR rating > 5.0 -> throw error
    IF reviewText is empty -> throw error
    reviewID    = new auto-generated int
    this.gameID  = gameID
    this.rating  = rating
    this.reviewText = reviewText
    this.datePosted = now()
    save new Review record to database
    update GameTitle.overallRating via getAverageRating()
    RETURN
update(rating, reviewText)
    IF rating < 0.0 OR rating > 5.0 -> throw error
    IF reviewText is empty -> throw error
    this.rating  = rating
    this.reviewText = reviewText
    save changes to database
    update GameTitle.overallRating via getAverageRating()
    RETURN
delete()
    delete this Review record from database

```



```

    update GameTitle.overallRating via getAverageRating()
    RETURN
flag()
    set review status to "Flagged"
    add this review to moderator content queue
    save changes to database
    RETURN

```

CLASS ForumPost

```

post(gameID, content, imageURL, subforum)
    IF gameID is null -> throw error
    IF content is empty -> throw error
    IF subforum is empty -> throw error
    postID      = new auto-generated int
    this.gameID = gameID
    this.content = content
    this.imageURL = imageURL (default "")
    this.subforum = subforum
    this.datePosted = now()
    save new ForumPost record to database
    RETURN
delete()
    delete this ForumPost record from database
    RETURN
flag()
    set post status to "Flagged"
    add this post to moderator content queue
    save changes to database
    RETURN
update(content, imageURL)
    IF content is empty -> throw error
    this.content = content
    this.imageURL = imageURL
    save changes to database
    RETURN

```

CLASS GameRequestForm

```

submit(gameTitle, developer, genre)
    IF gameTitle is empty -> throw error
    IF developer is empty -> throw error
    IF genre is empty -> throw error
    existing = look up GameTitle by gameTitle name
    IF existing found -> throw error (duplicate)
    formID    = new auto-generated int
    this.gameTitle = gameTitle
    this.developer = developer
    this.genre    = genre
    this.status   = "Submitted"
    save new GameRequestForm record to database
    RETURN
approve()
    IF this.status is not "Submitted" -> throw error

```

```

this.status = "Approved"
create new GameTitle using this.gameTitle, this.developer, this.genre
save new GameTitle to database
save form changes to database
RETURN

deny()
IF this.status is not "Submitted" -> throw error
this.status = "Denied"
save changes to database
RETURN

```

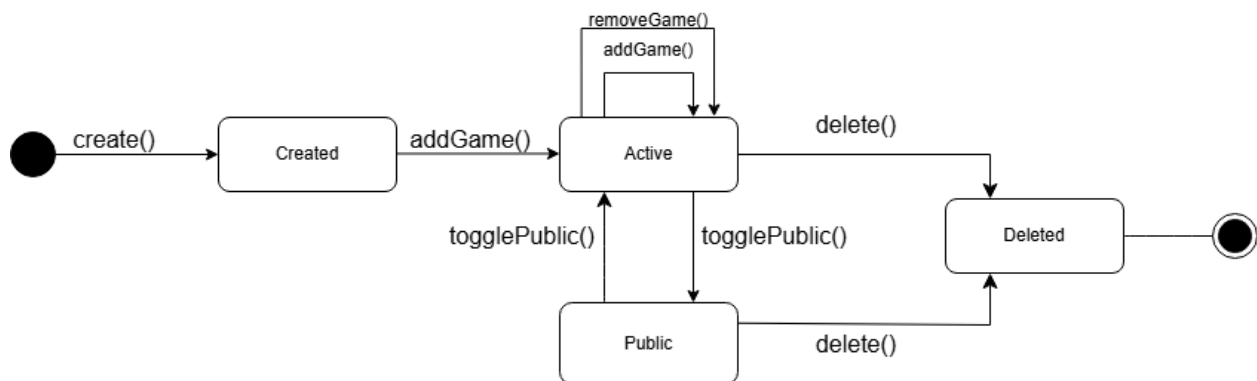
=====

END OF PSEUDOCODE

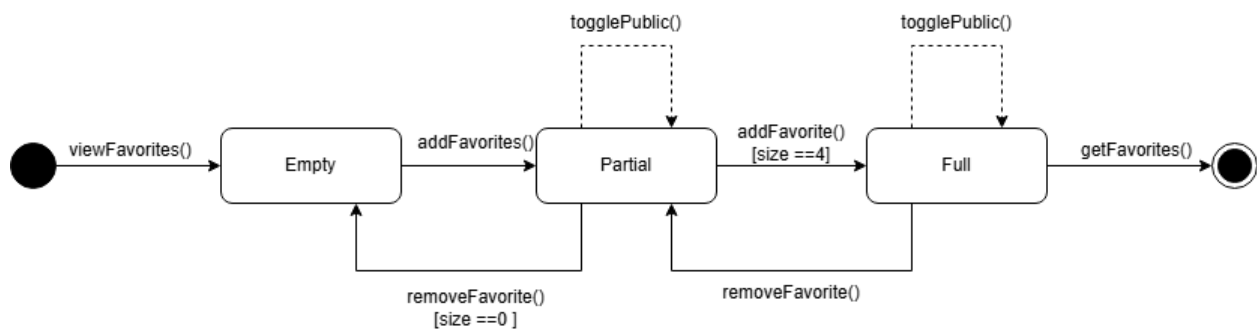
=====

Statechart Diagrams

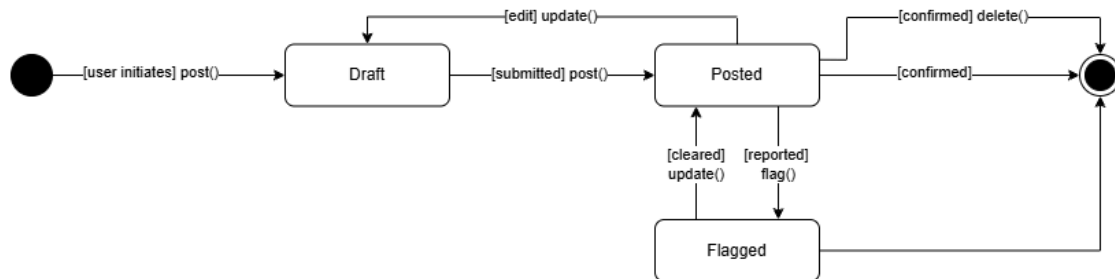
List



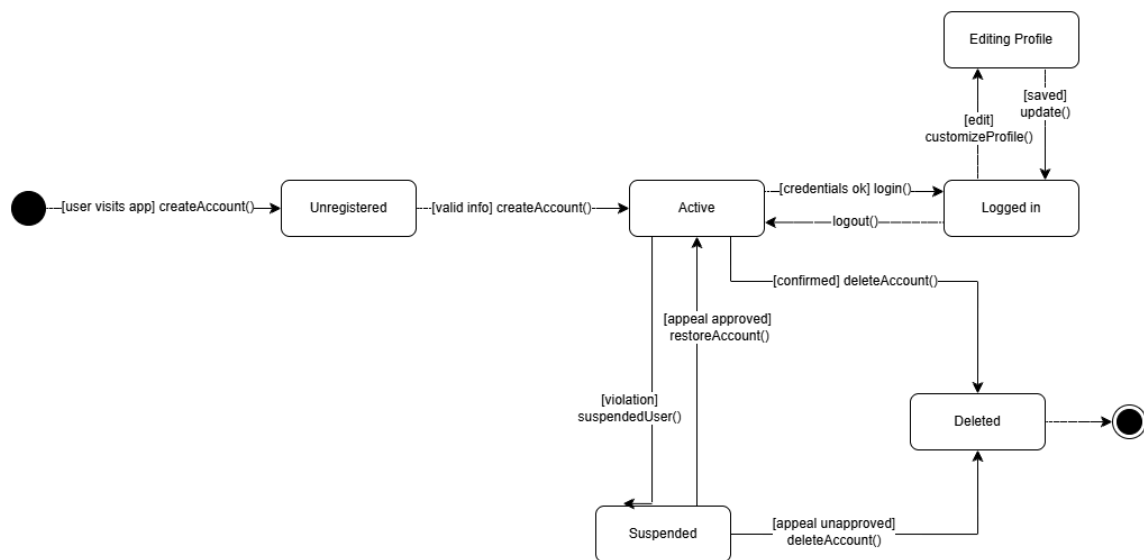
Favorites



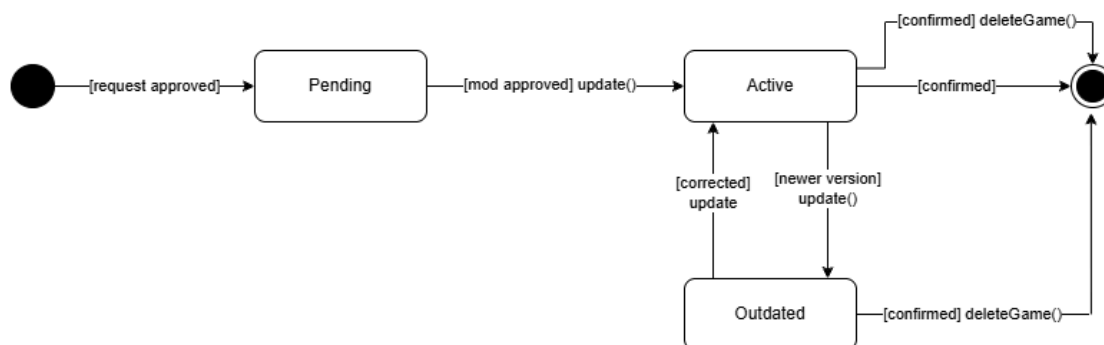
Review



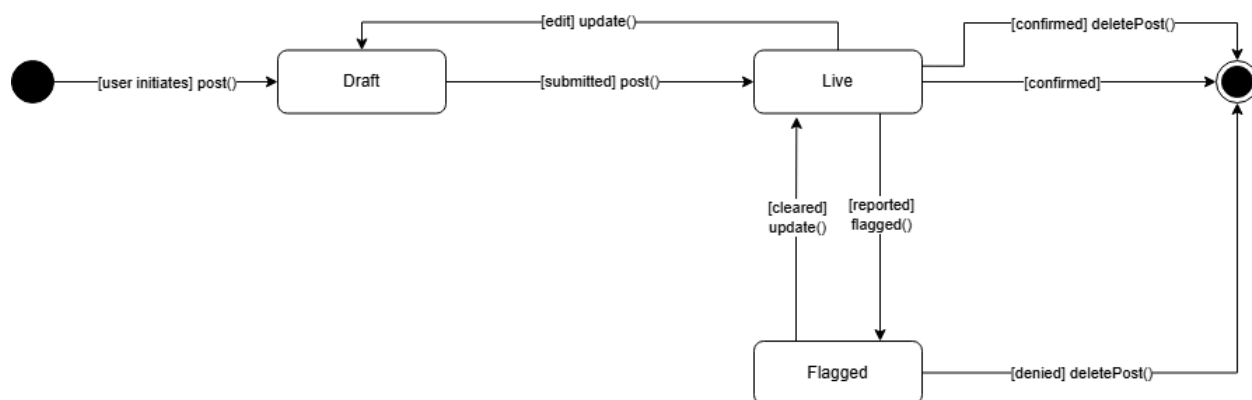
User



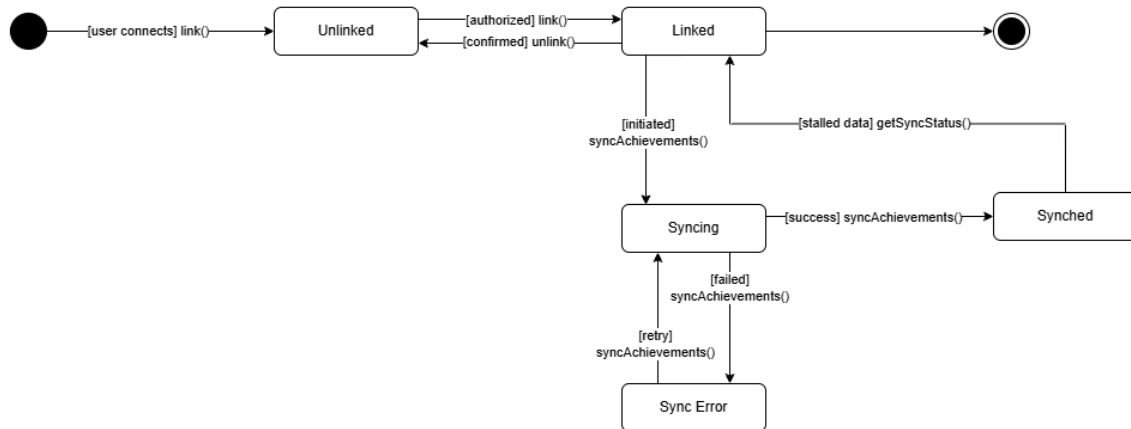
GameTitle



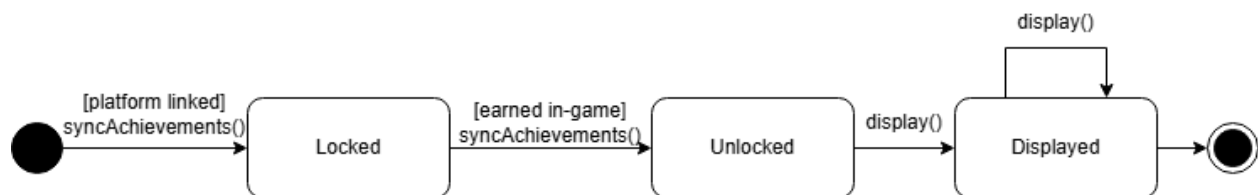
ForumPost



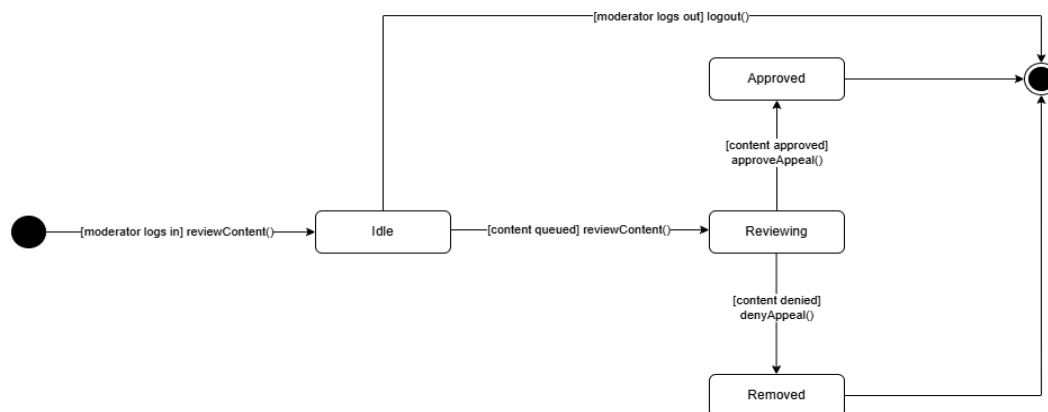
PlatformAccount



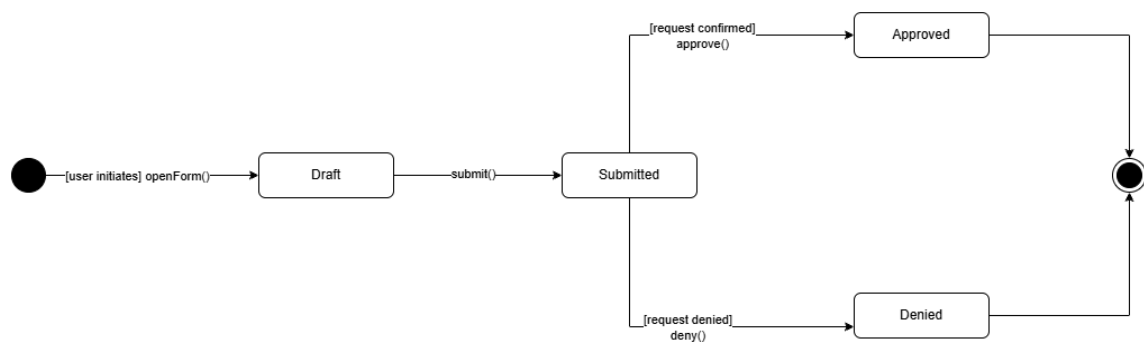
Achievement



Moderator

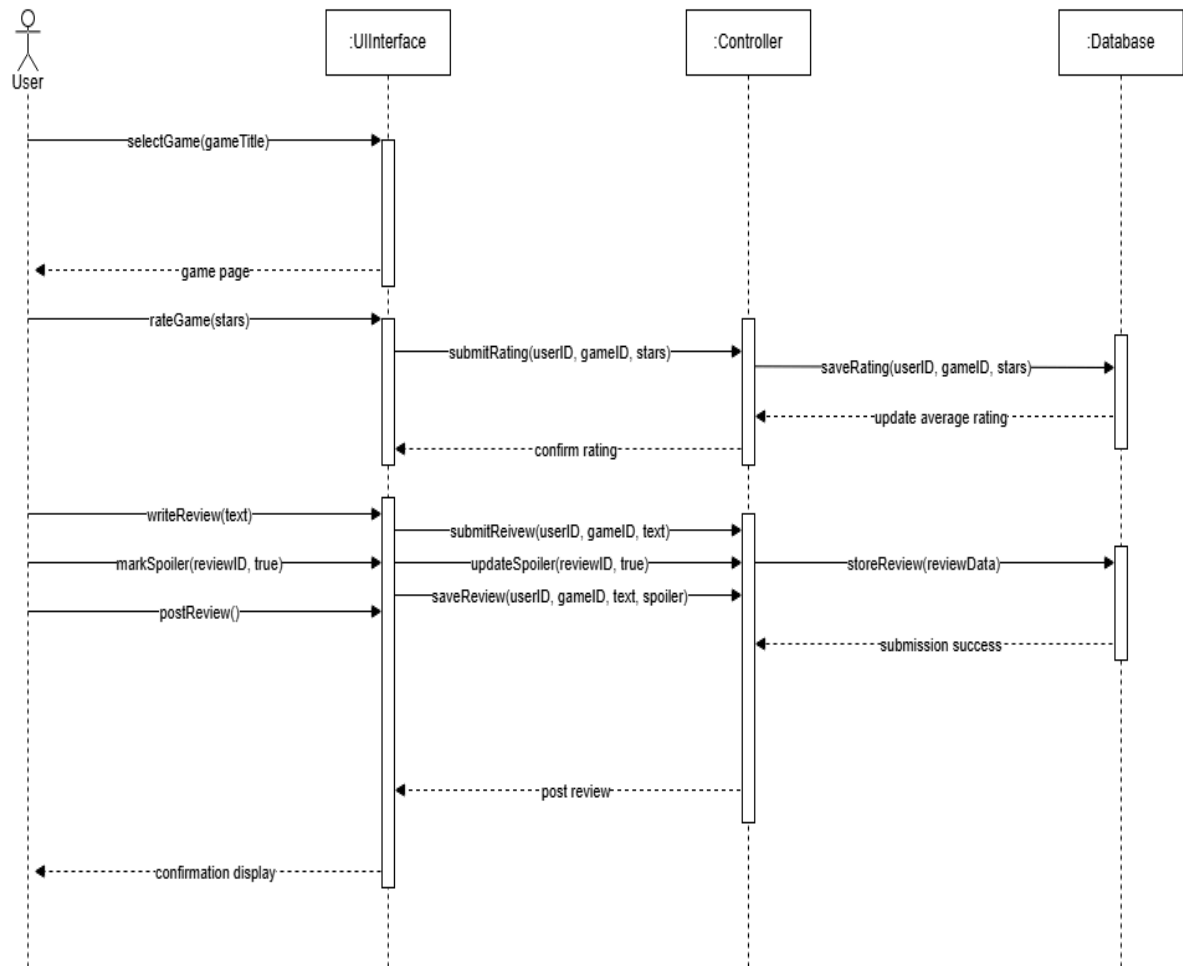


GameRequestForm

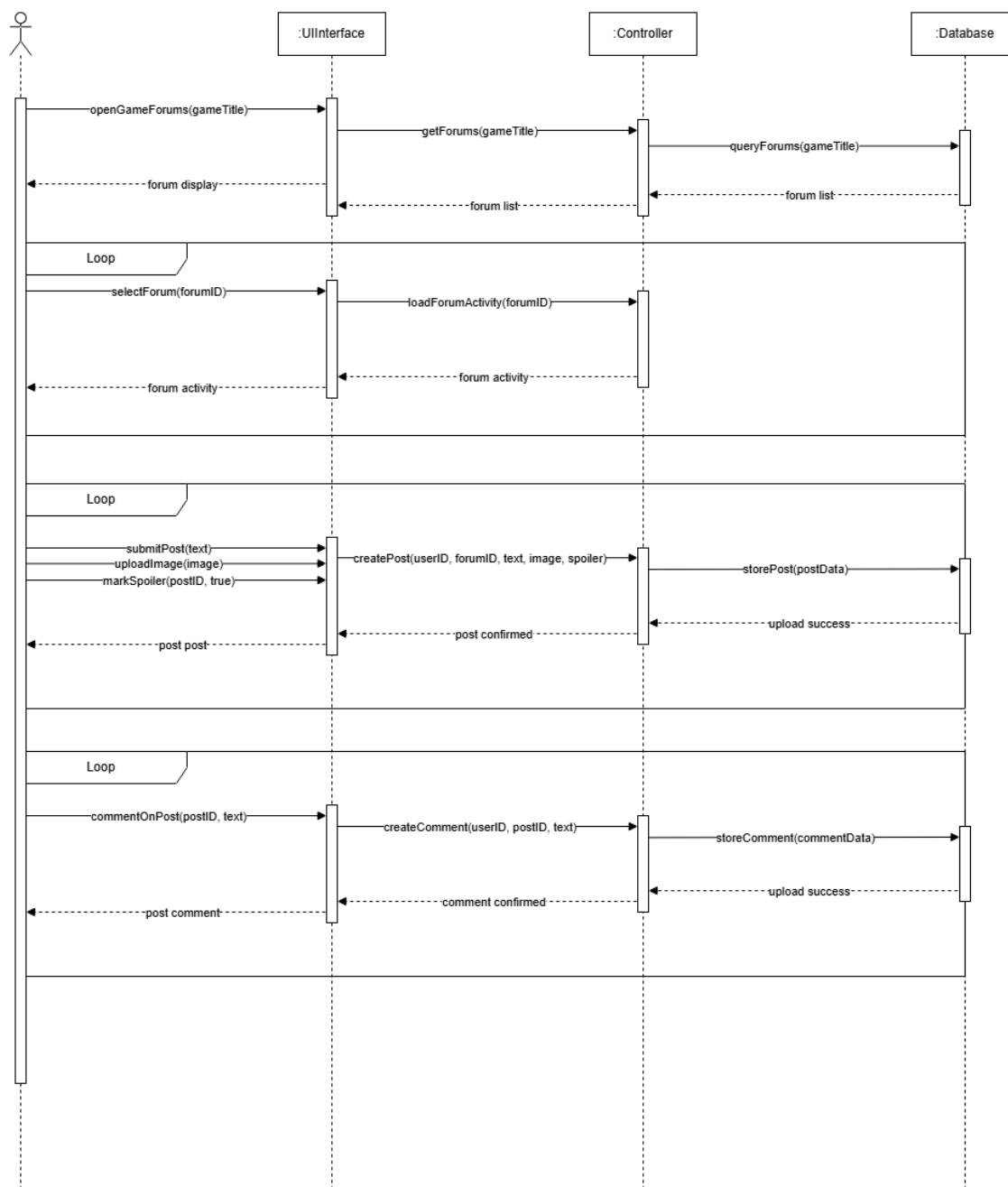


First Cut Domain Sequence Diagrams

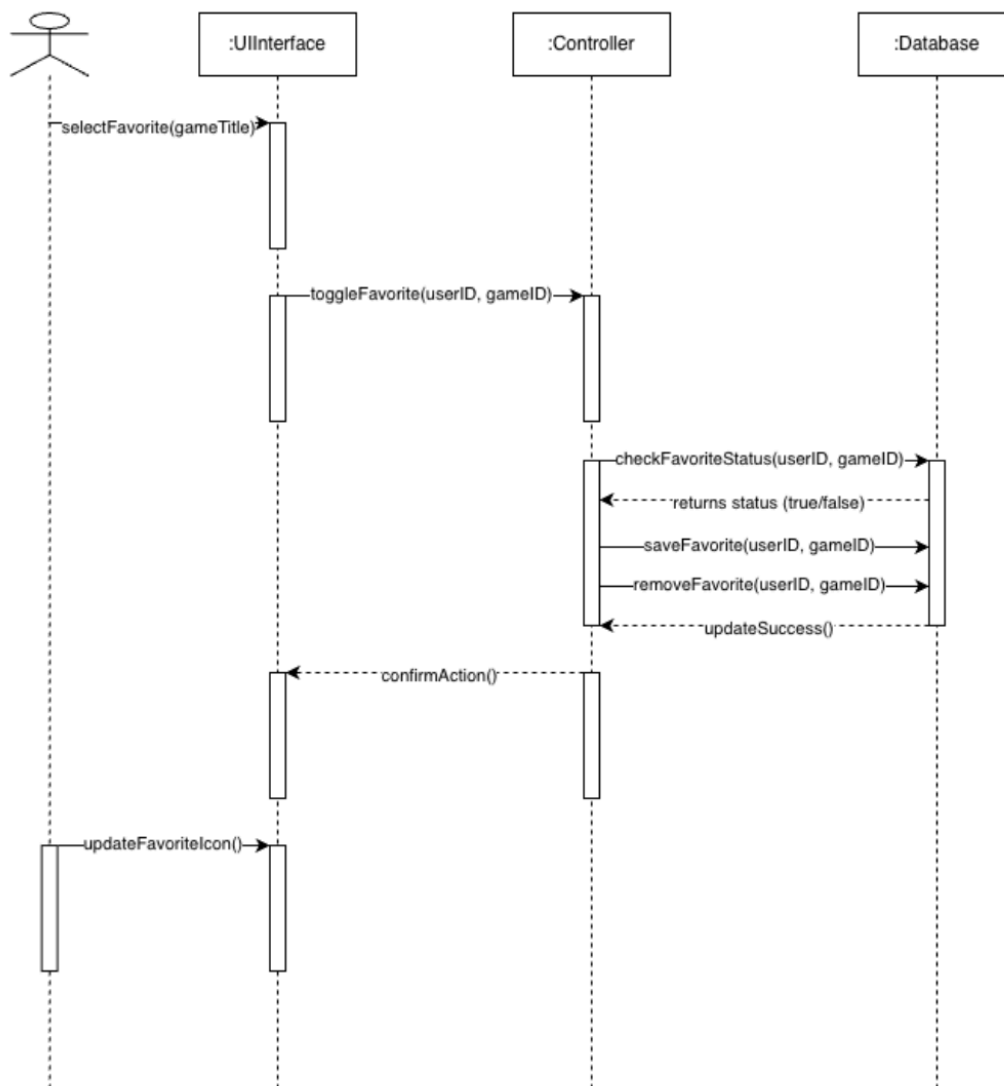
Rating/Reviewing



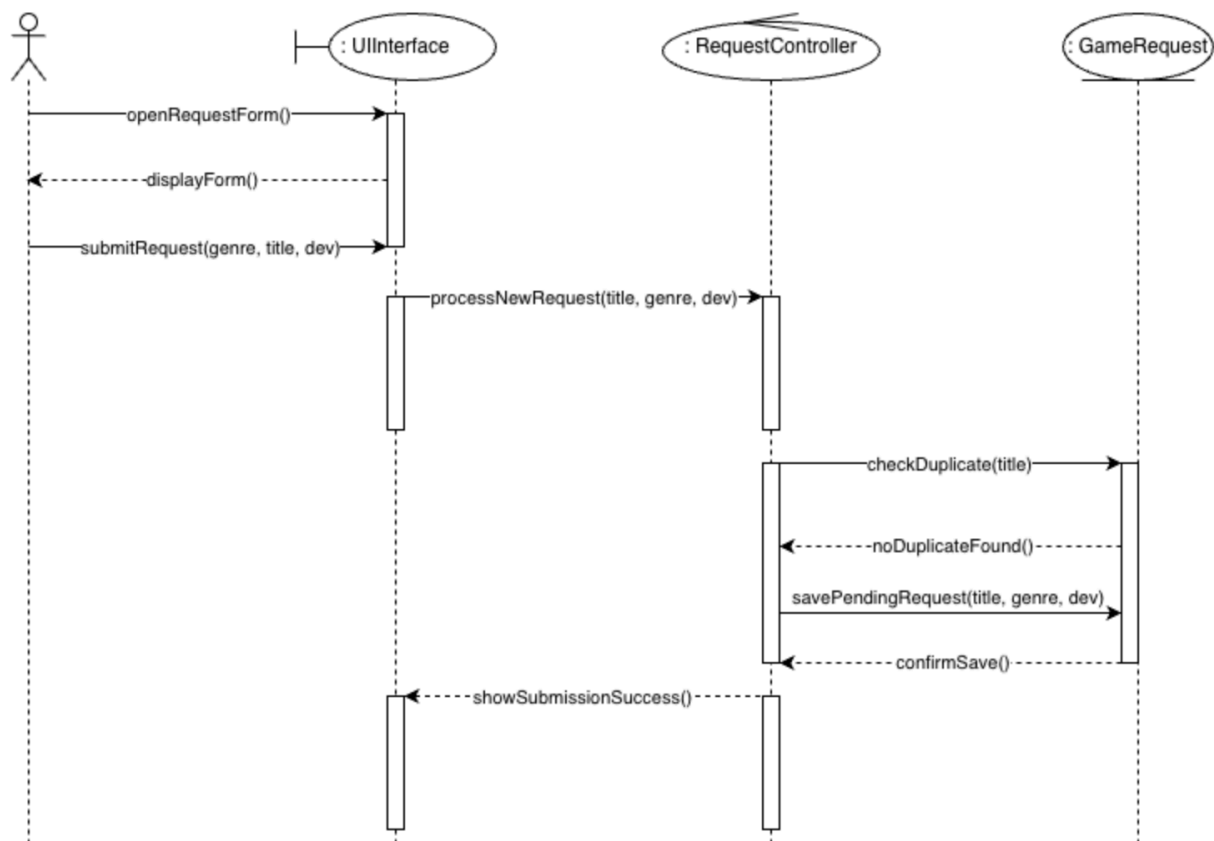
Game Forum Post



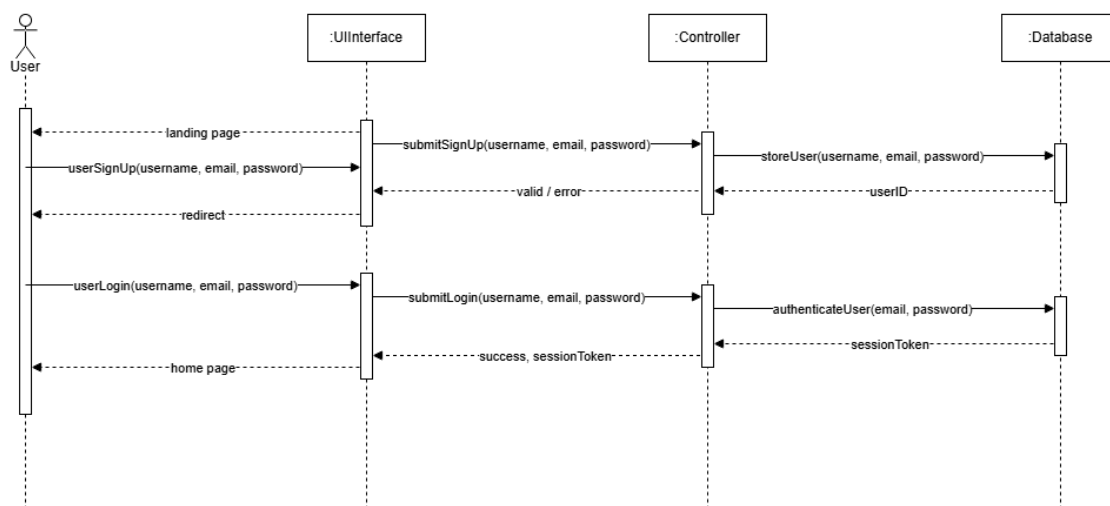
Favorites



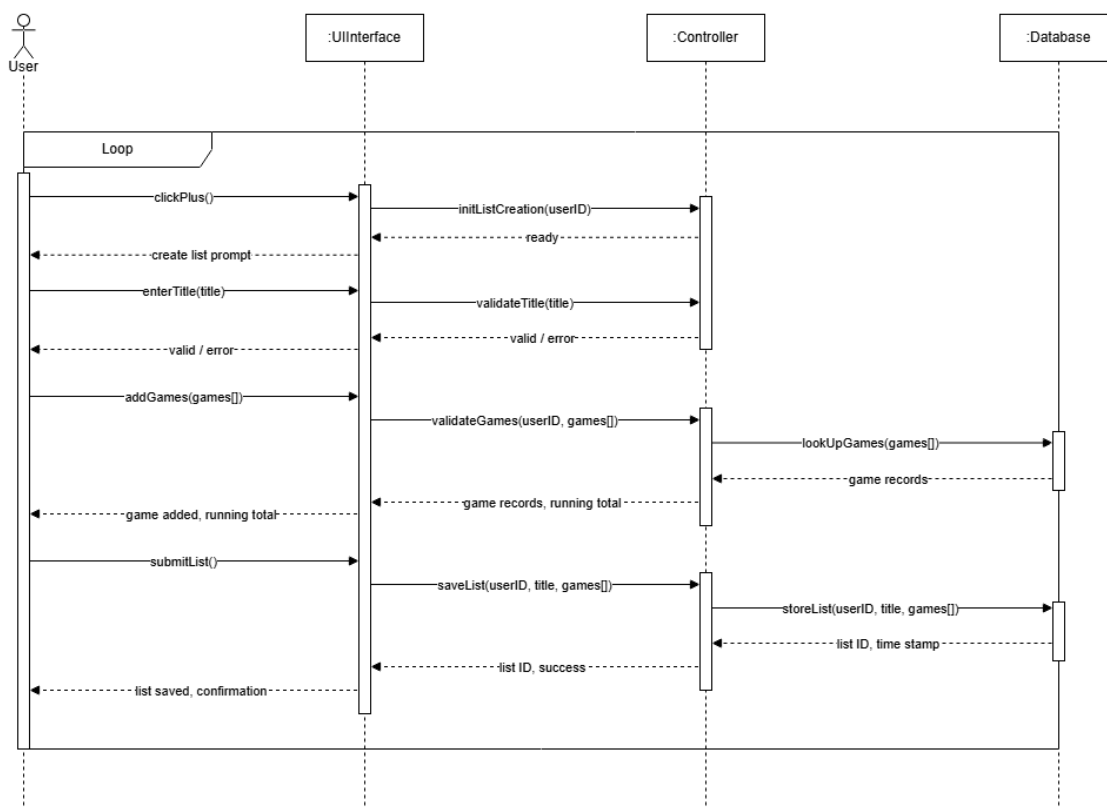
Game Request Form



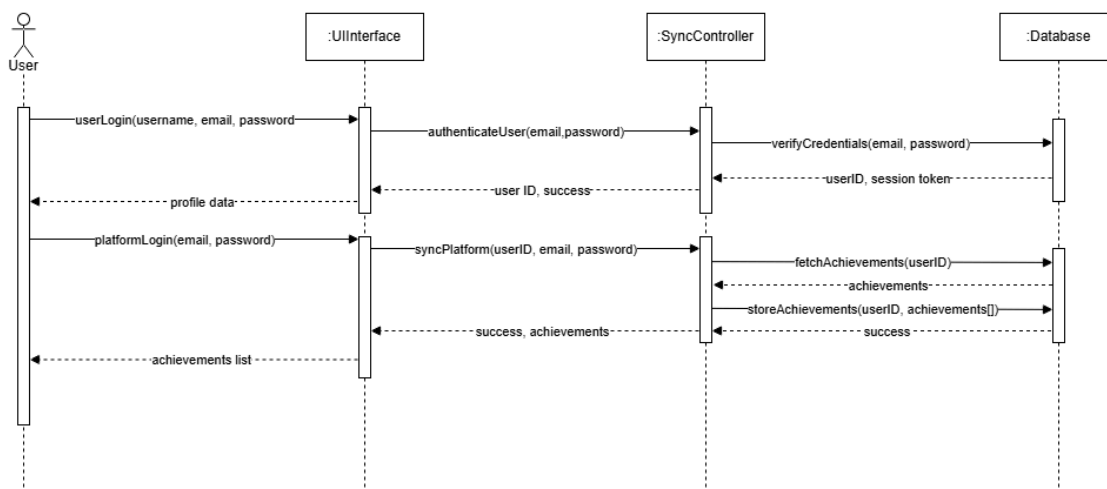
Account Creation



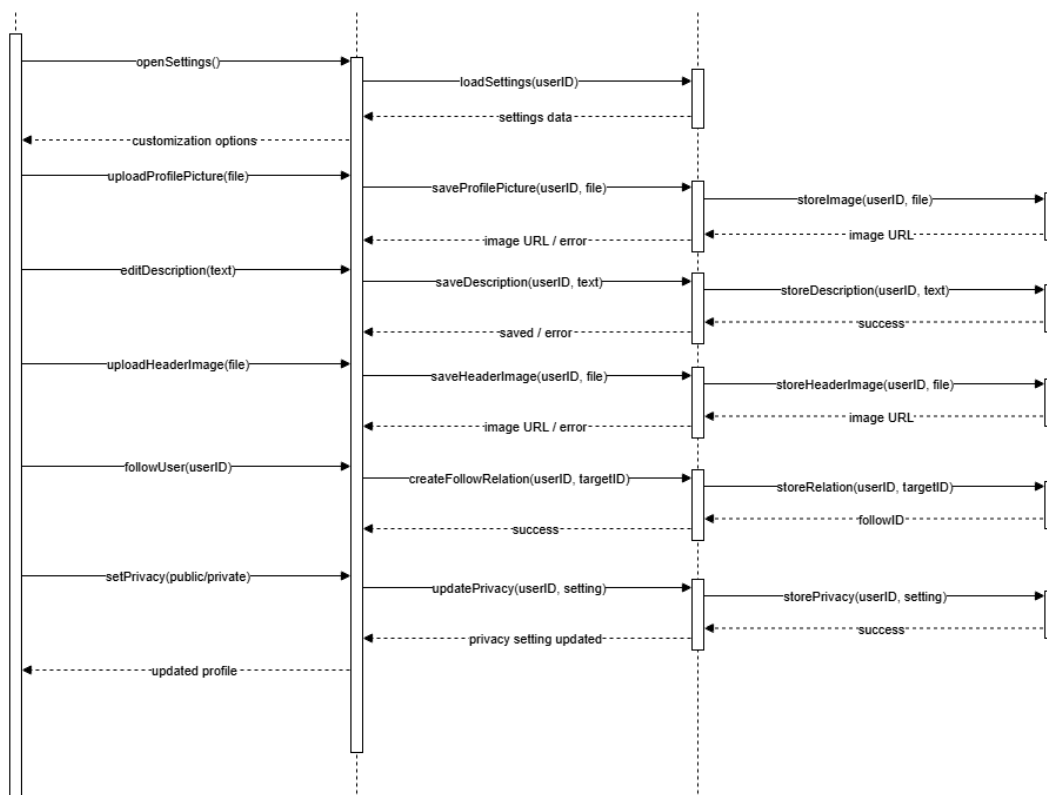
List creation



Achievement Sync



Account Customization



Moderator

